

Relatório Técnico Final

Sistema MeuBarbeiro

Projeto integrador da disciplina LDAMD
Backend .NET + SQLite + RabbitMQ + Flutter

Autor: Matheus Fernandes
Data: 29 de junho de 2026

1. Visão Geral

O MeuBarbeiro é um sistema distribuído para intermediar a relação entre clientes e barbeiros, com foco em busca de barbearias, solicitação de serviços, acompanhamento de status, aceite pelo prestador e avaliação final do atendimento. O projeto foi organizado em sprints cumulativas para atender aos requisitos acadêmicos da disciplina LDAMD e, ao mesmo tempo, construir um fluxo funcional de ponta a ponta entre backend, banco de dados, mensageira e aplicativos Flutter.

Do ponto de vista arquitetural, a solução combina um backend em C# .NET com organização em camadas, persistência em SQLite, mensageira RabbitMQ e dois aplicativos mobile: um voltado ao cliente final e outro ao prestador. O objetivo do relatório é consolidar as decisões técnicas, o desenho de arquitetura, os fluxos de negócio, os mecanismos assíncronos, os contratos REST, a modelagem de dados e as interfaces entregues ao final da Sprint 4.

2. Dominio de Negocio

O domínio do sistema é o agendamento de serviços em barbearias. O ator cliente navega por barbearias filtradas por cidade, consulta serviços oferecidos, escolhe uma data e horário e solicita o atendimento. A partir disso, o pedido deixa de ser apenas um cadastro síncrono e entra em um fluxo orientado a eventos. O barbeiro, por sua vez, possui uma conta autenticada, completa os dados da própria barbearia, cadastra serviços e gerencia os pedidos recebidos.

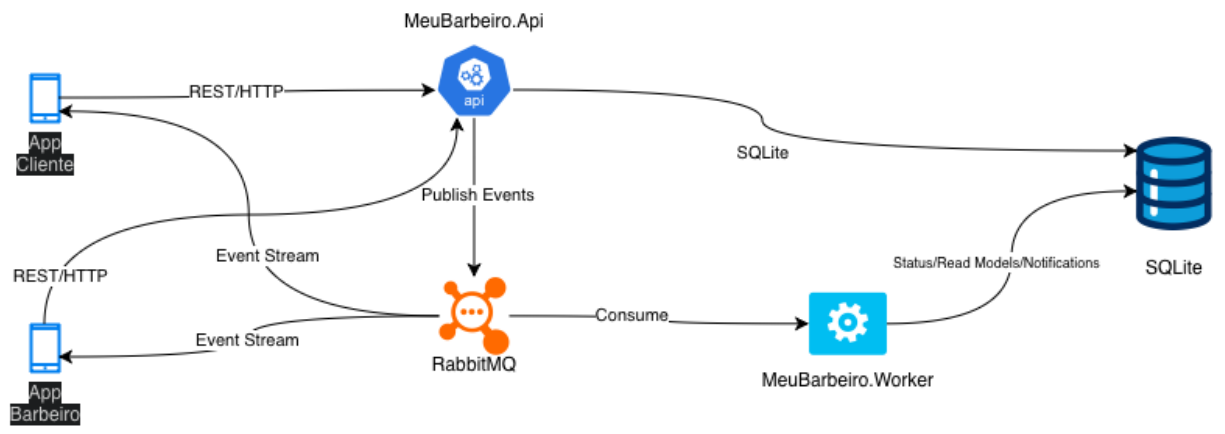
Na versão atual do produto, a regra adotada é de relacionamento 1:1 entre barbeiro e barbearia. Essa simplificação reduz ambiguidade no controle de autorização, facilita a validação do backend e torna o fluxo mais direto para a disciplina. Por isso, o cliente escolhe a barbearia e o backend resolve automaticamente o barbeiro responsável. O sistema também contempla a etapa de avaliação final, restrita a agendamentos concluídos e realizada exclusivamente com estrelas.

3. Arquitetura da Solução

A arquitetura do projeto segue uma divisão inspirada em Clean Architecture, com separação clara entre API, aplicação, domínio e infraestrutura. Essa organização foi importante para permitir a introdução progressiva de requisitos como JWT, RabbitMQ, worker assíncrono, auditoria de eventos e evolução do fluxo mobile sem acoplar a regra de negócio aos detalhes técnicos.

No backend, a API expõe contratos HTTP, a camada Application concentra casos de uso e DTOs, o Domain guarda entidades e enums, e a Infrastructure implementa repositórios, schema e mensageria. Paralelamente, um worker independente consome as filas RabbitMQ e executa reações assíncronas, como auditoria e mudança automática do status do pedido. Nos apps Flutter, o cliente e o prestador utilizam a mesma base REST, mas com roles, telas e casos de uso diferentes.

Diagrama consolidado de arquitetura:



4. Componentes do Sistema

Camada/Modulo	Responsabilidade	Tecnologias	Arquivos de referencia
MeuBarbeiro.Api	Expor endpoints REST, autenticar com JWT, aplicar autorizacao por perfil e delegar os casos de uso.	ASP.NET Core 9, Swagger, JWT Bearer	backend/src/MeuBarbeiro.Api/Program.cs; Controllers/*.cs
MeuBarbeiro.Application	Concentrar DTOs, contratos, services, regras de negocio e montagens de respostas.	.NET 9, FluentValidation.Results	backend/src/MeuBarbeiro.Application/Services/*.cs
MeuBarbeiro.Domain	Modelar entidades e enums do dominio como Appointment, Barber, Client, Barbershop, Review e ServiceOffering.	.NET 9	backend/src/MeuBarbeiro.Domain/Entities/*.cs
MeuBarbeiro.Infrastructure	Implementar persistencia SQLite, topologia RabbitMQ e repositórios concretos.	EF Core SQLite, RabbitMQ.Client	backend/src/MeuBarbeiro.Infrastructure/Persistence/*; Messaging/*
MeuBarbeiro.Worker	Consumir eventos, persistir auditoria e atualizar o status do agendamento para InProgress após processamento.	BackgroundService, RabbitMQ.Client	backend/src/MeuBarbeiro.Worker/Worker.cs
meu_barbeiro_cliente	Fluxo do cliente: autenticacao, busca de barbearias, agendamento, notificacoes visuais de status e avaliacao por estrelas.	Flutter, Dart, http, intl	mobile/apps/meu_barbeiro_cliente/lib/src/*
meu_barbeiro_prestador	Fluxo do barbeiro: autenticacao, completar barbearia, cadastrar serviços, acompanhar e decidir sobre pedidos.	Flutter, Dart, http, intl	mobile/apps/meu_barbeiro_prestador/lib/main.dart

5. Fluxos Funcionais Principais

Fluxo do cliente:

1. O usuário cria conta ou faz login com role Client.
2. O app lista barbearias por cidade.
3. O cliente acessa os serviços da barbearia.
4. O cliente escolhe os serviços desejados, data, horário e valor consolidado.
5. A API cria o agendamento com status Pending e publica o evento AppointmentRequested.
6. O worker consome o evento, registra auditoria e muda o agendamento para InProgress.
7. O app do prestador detecta o novo pedido "em análise".
8. O barbeiro aceita ou recusa, publicando AppointmentStatusUpdated.
9. O cliente recebe notificação visual no app a cada mudança de status.
10. Quando o atendimento é concluído, o cliente pode avaliar o agendamento com 1 a 5 estrelas.

Fluxo do prestador:

1. O barbeiro cria conta ou faz login com role Barber.
2. Completa a propria barbearia via PUT /barbershop/me.
3. Cadastra serviços vinculados a essa barbearia.
4. Visualiza os pedidos recebidos em polling quase em tempo real.
5. Decide entre aceitar ou recusar os pedidos em analise.

5. Endpoints REST Consolidados

Metodo e rota	Perfil	Finalidade	Observacoes
POST /api/v1/auth/register/client	Publico	Criar conta de cliente.	Retorna token JWT e dados basicos da sessao.
POST /api/v1/auth/register/barber	Publico	Criar conta de barbeiro.	Nao exige barbearia no momento do cadastro.
POST /api/v1/auth/login	Publico	Autenticar cliente ou barbeiro.	JWT inclui NamelIdentifier, Email e Role.
GET /api/v1/barbershop	Publico	Listar barbearias por cidade.	Consumido pelo app cliente.
PUT /api/v1/barbershop/me	Barber	Criar ou atualizar a propria barbearia.	Fluxo de onboarding do prestador.
GET /api/v1/barbershop/me	Barber	Consultar a barbearia vinculada ao barbeiro autenticado.	Suporta retorno ao formulario de perfil.
GET /api/v1/services?barbershopId=...	Publico	Listar servicos da barbearia.	Consumido no fluxo de agendamento do cliente.
POST /api/v1/services	Barber	Cadastrar servicos da propria barbearia.	Valida ownership pelo token.
POST /api/v1/appointment	Client	Criar agendamento com servicelds.	BarberId e resolvido automaticamente por BarbershopId.
GET /api/v1/appointment/mine	Client/Barber	Listar agendamentos do usuario autenticado.	Retorna dados enriquecidos e review.
PATCH /api/v1/appointment/{id}/status	Barber	Aceitar ou recusar pedido.	Usado pelo app prestador.
POST /api/v1/appointment/{id}/review	Client	Avaliar um atendimento concluido com 1 a 5 estrelas.	Bloqueado para status diferentes de Completed.
GET /api/v1/event-processing-audits	Temporario	Auditoria dos eventos processados.	Apoio para demonstracao academica e troubleshooting.

6. Modelo de Dados e Persistencia

O sistema utiliza SQLite como banco relacional. A configuração base fica em `backend/src/MeuBarbeiro.Api/appsettings.json`, com o arquivo `meubarbeiro.db`. A inicializacao do schema acontece em `DatabaseSchemaInitializer`, abordagem adotada para simplificar a execucao em ambiente academico sem dependencia de migrations formais.

Tabela	Campos essenciais	Observacoes de negocio
Users	Id, Name, Email, PasswordHash, Role, CreatedAt	Base de autenticacao e autorizacao por perfil.
Clients	Id, UserId	Relaciona a conta autenticada ao ator cliente.
Barbers	Id, UserId, BarbershopId	No escopo atual, 1 barbeiro para 1 barbearia.
Barbershops	Id, Name, City, Address, Description, AverageRating	Perfil operacional do prestador e nota media acumulada.
ServiceOfferings	Id, BarbershopId, Name, Price, Description, DurationMinutes	Catalogo de servicos oferecidos pela barbearia.
Appointments	Id, ClientId, BarberId, BarbershopId, ScheduledAtUtc, TotalPrice, Status	Centro do fluxo de negocio do sistema.
AppointmentServiceSelections	Id, AppointmentId, ServiceOfferingId	Persistencia dos servicos efetivamente escolhidos no pedido.
Reviews	Id, AppointmentId, ClientId, BarberId, BarbershopId, Stars, CreatedAtUtc	Somente uma avaliacao por agendamento concluido.
EventProcessingAudits	Id, EventName, QueueName, Payload, ProcessedAtUtc, Status, ErrorMessage	Rastro de processamento assincrono para debugging e evidencia.

7. Mensageria e Processamento Assíncrono

O projeto utiliza RabbitMQ como middleware orientado a mensagens. A API publica eventos de integração sempre que um agendamento e criado ou sofre mudanca de status, enquanto o worker atua como consumidor. A topologia escolhida e baseada em uma exchange `meu-barbeiro.events`, com filas distintas para `appointments.requested` e `appointments.status-updated`.

Quando o cliente cria um agendamento, a API persiste os dados no SQLite e publica `AppointmentRequestedIntegrationEvent`. O worker consome essa mensagem, persiste um registro na tabela `EventProcessingAudits` e atualiza o agendamento para `InProgress`. Na interface, esse estado foi renomeado para “Em analise”, sinalizando que o pedido chegou ao prestador e esta aguardando decisão.

Quando o barbeiro aceita ou recusa o pedido, a API altera o status do agendamento e publica `AppointmentStatusUpdatedIntegrationEvent`. O processamento desse evento tambem gera

auditoria. O resultado prático é um fluxo real de assincronicidade: a criação do agendamento não encerra o processo, mas dispara reações em outro processo (worker) e reflexos visuais nas duas interfaces mobile.

8. Segurança, Autenticação e Autorização

O backend utiliza autenticação com JWT Bearer. O token gerado no login ou registro inclui `Namelfidifier`, `Name`, `Email` e `Role`. A API aplica autorização por perfil nas actions, por exemplo: apenas clientes podem criar agendamentos e enviar reviews; apenas barbeiros podem completar a barbearia, cadastrar serviços e atualizar o status de um pedido.

Essa separação protege o domínio e evita fraude por manipulação manual de ids no front. O cliente não envia ``Clientld``; o backend o resolve pelo token. O barbeiro não manipula pedidos de outra barbearia, pois a API busca o perfil autenticado e valida `ownership`. Essa foi uma evolução importante do projeto ao longo das sprints, substituindo ids “fictícios” passados pelo request por contexto autenticado extraído diretamente do JWT.

9. Aplicativo do Cliente

O app Flutter do cliente implementa o fluxo completo de descoberta, solicitação, acompanhamento e avaliação. A camada ``backend_api.dart`` centraliza a comunicação com o backend e os modelos JSON. A tela principal oferece três áreas: explorar barbearias, acompanhar agenda e gerenciar a conta.

Na etapa de agendamento, o cliente seleciona serviços, data e horário, e o app envia ``barbershopld``, ``serviceids``, ``scheduledAtUtc`` e ``totalPrice``. Depois disso, a agenda passa a exibir respostas enriquecidas com nome da barbearia, serviços selecionados, valor total e status. O app também possui polling leve para detectar mudanças de status, exibindo notificações visuais sempre que um pedido muda de estado. Quando o agendamento está concluído e ainda não possui review, surge a ação “Avaliar atendimento”, abrindo um diálogo de 1 a 5 estrelas.

10. Aplicativo do Prestador

O app Flutter do prestador possui três marcos funcionais: autenticação, completar a barbearia e operar o dashboard. A primeira tela cuida do login e cadastro do barbeiro. A seguir, o fluxo direciona para ``PUT /barbershop/me``, permitindo salvar ou atualizar os dados da barbearia. Com o perfil operacional pronto, o barbeiro entra no dashboard com abas de pedidos, serviços e perfil.

Na aba de pedidos, o app busca ``GET /appointment/mine`` e mostra status, cliente, barbearia, serviços e valor total. Como o worker muda o status para ``InProgress`` assim que processa a mensagem do novo pedido, o prestador enxerga pedidos “Em análise” sem depender de refresh manual constante. Para reforçar a percepção de chegada de novos pedidos, o app também faz polling a cada oito segundos e mostra banners/snackbars quando identifica novos agendamentos em análise.

11. Qualidade, Testes e Validações

O projeto foi validado continuamente com build e testes automatizados. O backend possui uma bateria de testes unitários concentrados em `AppointmentService`, cobrindo criação de agendamento, listagens por papel, filtros de status, publicação de eventos, validação de services selecionados e regras de review. Ao final da consolidação atual, o conjunto estava com 16 testes passando.

No frontend, os apps Flutter foram validados com `flutter analyze`, e os contratos REST foram exercitados via Swagger e chamadas HTTP nos fluxos de autenticação, serviços, barbearia, agendamento e review. No aspecto de mensageria, o comportamento do worker é observável tanto pelos logs quanto pelos registros na tabela `EventProcessingAudits`, o que também serve como evidência acadêmica do MOM funcionando.

12. Limitações e Evoluções Futuras

Apesar de funcional e aderente ao escopo da disciplina, o sistema ainda possui espaços claros para evolução. O modelo atual assume 1 barbeiro por barbearia. Uma versão de produção provavelmente precisaria suportar vários barbeiros, agenda individual por profissional e disponibilidade real baseada em grade horária. Da mesma forma, as notificações atualmente são visuais com polling; em contexto de produto maduro, WebSockets, SignalR, push notifications ou filas dedicadas a notificações seriam caminhos naturais.

Do ponto de vista de persistência, o projeto utiliza `DatabaseSchemaInitializer` para simplificar a distribuição local com SQLite. Em um ambiente mais robusto, seria desejável migrar para migrações formais do EF Core. Também seriam recomendadas trilhas adicionais de testes de integração, testes end-to-end entre API/worker/RabbitMQ e maior cobertura do app prestador.

11. Referências Técnicas do Projeto

Esta seção consolida os arquivos mais relevantes consultados para a construção e manutenção do sistema. As referências abaixo servem como mapa de navegação do código-fonte e como evidências técnicas da implementação.

Área	Arquivos de referência
Bootstrap da API	backend/src/MeuBarbeiro.Api/Program.cs
Autenticação e JWT	backend/src/MeuBarbeiro.Api/Controllers/AuthController.cs; backend/src/MeuBarbeiro.Application/Services/AuthService.cs; backend/src/MeuBarbeiro.Application/Services/JwtTokenService.cs
Agendamentos	backend/src/MeuBarbeiro.Api/Controllers/AppointmentController.cs; backend/src/MeuBarbeiro.Application/Services/AppointmentService.cs

Barbearias e servicos	backend/src/MeuBarbeiro.Api/Controllers/BarbershopController.cs; backend/src/MeuBarbeiro.Api/Controllers/ServicesController.cs; backend/src/MeuBarbeiro.Application/Services/BarbershopService.cs; backend/src/MeuBarbeiro.Application/Services/ServicesService.cs
Mensageria	backend/src/MeuBarbeiro.Infrastructure/Messaging/*.cs; backend/src/MeuBarbeiro.Contracts/Events/*.cs
Worker e auditoria	backend/src/MeuBarbeiro.Worker/Worker.cs; backend/src/MeuBarbeiro.Api/Controllers/EventProcessingAuditController.cs
Persistencia SQLite	backend/src/MeuBarbeiro.Infrastructure/Persistence/AppDbContext.cs; Configurations/*.cs; Repositories/*.cs; DatabaseSchemaInitializer.cs
Cliente Flutter	mobile/apps/meu_barbeiro_cliente/lib/src/data/backend_api.dart; mobile/apps/meu_barbeiro_cliente/lib/src/features/client_app_page.dart
Prestador Flutter	mobile/apps/meu_barbeiro_prestador/lib/main.dart
Documentacao previa	README.md; docs/Sprint 1/Documento de Proposta/Proposta_MeuBarbeiro_Sprint1.pdf; docs/Sprint 2/documentacao-eventos.md; docs/Sprint 2/relatorio-integracao.md; docs/Sprint 2/evidencia-execucao.md

13. Conclusão

O MeuBarbeiro evoluiu de uma proposta de domingo para um sistema distribuido funcional, integrando backend REST, persistência SQLite, mensageria RabbitMQ, processamento assíncrono e dois aplicativos Flutter com fluxos distintos. O resultado final atende aos objetivos centrais da disciplina: dois perfis de usuario bem definidos, operações essenciais do dominio, banco documentado, MOM funcionando, produtor e consumidor de eventos, demonstração de assincronicidade e interfaces capazes de refletir essas mudancas ao usuario final.

Mais do que cumprir os requisitos minimos, a solucao consolidou principios importantes de engenharia de software: separação de responsabilidades, autorização orientada a identidade, consistencia entre dominio e interface, auditoria de eventos e evolução incremental por sprint. Com isso, o projeto oferece uma base tecnicamente coerente, demonstravel e extensivel para futuras iterações.